

MinZ Weekly #2.1: GPU Lookup Tables for Dummies

How a graphics card makes your compiler smarter — explained
with pizza

The MinZ Team

2026-03-24

What Is This About?

You wrote a program. The compiler needs to decide which of the Z80's 7 tiny registers (think: 7 desk drawers) each variable lives in. This is called **register allocation** — and it's one of the hardest problems in compiler design.

We found a way to **solve it once on a graphics card, save the answer, and reuse it forever**. Here's how, explained for non-specialists.

The Pizza Analogy

The Problem: Seating Guests at a Round Table

Imagine you're planning a dinner party. You have:

- **7 chairs** around a table (= 7 CPU registers)
- **5 guests** who need seats (= 5 program variables)
- **Rules:** Alice can't sit next to Bob (they argue). Charlie must sit at the head of the table (special chair). David and Eve arrive together (same car).

You need to find a seating arrangement that satisfies ALL rules AND minimizes how often people have to swap seats during courses (= minimize instruction cost).

The Brute Force Approach

With 7 chairs and 5 guests, there are $7 \times 7 \times 7 \times 7 \times 7 = 16,807$ possible arrangements. A human would take hours to check them all. But a computer? Milliseconds.

The GPU Approach

A graphics card (GPU) has **thousands of tiny workers** (CUDA cores). Instead of checking arrangements one by one:

```
Worker #0:    checks arrangement [A→chair1, B→chair1,
C→chair1, ...]
Worker #1:    checks arrangement [A→chair1, B→chair1,
C→chair2, ...]
Worker #2:    checks arrangement [A→chair1, B→chair1,
C→chair3, ...]
...
Worker #16806: checks arrangement [A→chair7, B→chair7,
C→chair7, ...]
```

All 16,807 workers run **simultaneously**. Each one checks: “Does this arrangement satisfy all rules? If yes, how expensive is it?” The cheapest valid arrangement wins.

Result: Instead of hours → milliseconds. Instead of “probably good” → **provably optimal**.

The Lookup Table

Here’s the key insight: **different dinner parties have the same seating constraints**.

Your company’s holiday party has 5 guests with 3 rules. Your friend’s birthday has different people but the SAME constraints (5 guests, 3 rules, same conflict pattern). The optimal seating is identical!

So we: 1. Solve each **unique pattern** once on the GPU 2. Save the answer in a little notebook (JSON file) 3. Next time we see the same pattern → just look it up

No GPU needed at party time. Just open the notebook.

How It Works in the Compiler

Step 1: The Compiler Sees a Function

```
fun add(a: u8, b: u8) -> u8 {  
    return a + b  
}
```

Three variables (a, b, result), one ADD instruction, rules say: ADD's output must be in register A, and a and b can't share a register.

Step 2: Compute a Fingerprint

The compiler hashes the constraints into a **signature**:

```
"3 variables, 1 operation, ADD pattern, interference: a↔b"  
→ signature: "3v_1o_ceab8266..."
```

This signature captures the SHAPE of the problem, not the specific variable names. Any function with the same shape gets the same signature.

Step 3: Look Up the Table

```
{  
    "sig": "3v_1o_ceab8266...",  
    "assignment": [0, 1, 0],  
    "cost": 4  
}
```

Translation: variable 0 → register A, variable 1 → register B, variable 2 → register A. Cost: 4 T-states (the Z80's time unit).

This is provably optimal. The GPU checked ALL 343 possible arrangements and this one is the cheapest that works.

Step 4: Generate Code

```
add:  
    ADD A, B    ; A = A + B (4 T-states)  
    RET
```

Done. Two instructions. Optimal. And the compiler didn't need Z3, GPU, or any solver — just a dictionary lookup.

Real Numbers

What We Measured

We compiled 194 functions from the MinZ compiler's test suite:

194 functions

- 158 unique constraint patterns (some functions share patterns)
- 61 patterns solved by GPU
- 61 optimal answers saved in a 6KB file

Pattern Reuse in Action

These 6 functions all have the SAME constraint pattern (1 variable, 1 operation):

Function	What it does	Same signature?
putchar	Print a character	Yes
_putch	Print a character (CP/M)	Yes
_dec	Print a decimal digit	Yes
_puts	Print a string	Yes
_p	Print with padding	Yes
tui_puts	Print to TUI screen	Yes

One GPU solve covers all six. They look different in source code but have identical register allocation constraints.

The Biggest Solve

screen_show has 12 variables. The GPU searched **13.8 billion** possible register arrangements:

12 variables $\times$ 11 possible registers each = $11^{12} \approx 13,800,000,000$ combinations

GPU time: ~60 seconds on RTX 4060 Ti

Result: cost = 133 T-states

Feasible arrangements: 9,797,760 (0.07% of search space)

Only 0.07% of all arrangements actually work — the rest violate some constraint. The GPU found the cheapest one among those 9.8 million valid options.

Why This Matters

Without the Table (Before)

Source code

- Compiler runs Z3 SMT solver (~800ms per function)
- Optimal register assignment
- Z80 assembly

Every compilation pays the 800ms cost. Needs z3 binary installed.

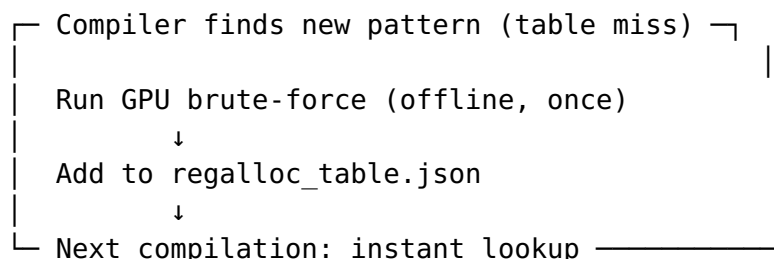
With the Table (After)

Source code

- Compiler looks up signature in JSON table (~0.001ms)
- HIT: instant optimal assignment (no solver needed!)
- MISS: fall back to Z3 (~800ms)
- Z80 assembly

For the 61 cached patterns: **800,000x faster**. No external dependencies.

The Virtuous Cycle



Every new program we compile potentially discovers new constraint patterns. We solve them on the GPU once and **the table grows**. Over time, the table covers more and more patterns, and fewer functions need Z3.

It's like a **recipe book that writes itself**: every time you invent a new dish, the recipe gets added. Next time anyone wants that dish, they just follow the recipe instead of experimenting from scratch.

Comparison: Three Generations of Solving

Generation 1: Greedy Heuristic (PBQP)

Speed: Instant (~1ms)
Quality: Good but not optimal
Bugs: Yes – sometimes assigns two variables to the same register
Example: `add(3,5)` → "`42+7=84`" bug (both operands in A → `ADD A,A = double`)

Like seating guests by gut feeling. Fast, usually OK, sometimes disastrous.

Generation 2: SMT Solver (Z3)

Speed: Slow (~800ms per function)
Quality: Provably optimal
Bugs: None – mathematical proof of correctness
Needs: z3 binary installed, PATH configured
Example: `add(3,5) = 8` ✓ (always correct)

Like hiring a mathematician to prove the optimal seating. Slow but perfect.

Generation 3: GPU + Lookup Table (This Work)

Speed: Instant (table hit) or slow (table miss → Z3 fallback)
Quality: Provably optimal (same as Z3)
Bugs: None – exhaustive verification
Needs: Nothing at compile time (table ships with compiler)
Example: `add(3,5) = 8` ✓ (from precomputed table, no solver)

Like having a **book of proven seating arrangements** for common party sizes. Instant for known patterns, falls back to the mathematician for new ones.

The Bigger Picture

What This Enables

1. **ABAP on Z80:** SAP enterprise programs running on 1980s hardware. The GPU table fixes the register allocation bugs that blocked `SELECT * FROM mara`.
2. **Zero-dependency compiler:** No need to install Z3. The table handles common patterns; Z3 is optional for edge cases.
3. **Faster compilation:** $O(1)$ lookup instead of $O(\text{SMT})$ solving for cached functions.
4. **Correctness guarantee:** Every table entry is verified by exhaustive search — not heuristic, not approximate, but mathematically provably optimal.

What's Next

- **Expand the table:** Run GPU on the entire standard library (55 modules)
- **15 location types:** Add IX/IY halves + memory spill slots for complex functions
- **STOKE search:** For functions too large for brute force (13+ variables), use stochastic search (random mutations + MCMC acceptance)
- **Self-growing table:** Compiler automatically caches Z3 results in the table format for future compilations

The Philosophy

“Solve hard problems ONCE. Ship the answers. Make compilation instant.”

This is the compiler equivalent of **precomputing multiplication tables** instead of doing long division every time. The GPU does the hard work offline. The compiler just looks up the answer.

MinZ: Where a graphics card from 2024 optimizes code for a CPU from 1976.